

# Practical Work 5: The Longest Path

Group ID: 16

December 16, 2025

## 1 Introduction

This report outlines the implementation of a MapReduce system designed to solve the "Longest Path" problem. The objective is to process a large dataset of file paths (simulated from the `find` / command) and identify the single path with the maximum character length using a distributed computing approach.

## 2 System Implementation

### 2.1 Choice of Framework

We utilized the **Python Concurrent Futures** module (`concurrent.futures`) for this implementation.

- **ProcessPoolExecutor:** This class allows us to create a pool of worker processes. Unlike standard threading, this bypasses the Global Interpreter Lock (GIL), enabling true parallel processing on multi-core CPUs, which is essential for CPU-bound tasks like string comparison in large datasets.
- **Simplicity:** It provides a high-level interface for asynchronously executing callables, making the MapReduce code cleaner and more readable compared to low-level multiprocessing.

### 2.2 Architecture Design

The system architecture follows a vertical data processing flow, reducing the data volume at each stage.

### 2.3 Code Logic

**Mapper Phase:** Each worker receives a chunk of file paths. Instead of outputting every path, it optimizes network usage by calculating the local maximum length internally.

```
1 @staticmethod
2 def mapper(paths):
3     if not paths: return ""
4     return max(paths, key=len)
```

**Reducer Phase:** The reducer receives only the "champions" (local maximums) from each worker and performs a final comparison to determine the global maximum.

```
1 @staticmethod
2 def reducer(local_maxima):
3     return max(local_maxima, key=len)
```

## 3 Task Distribution

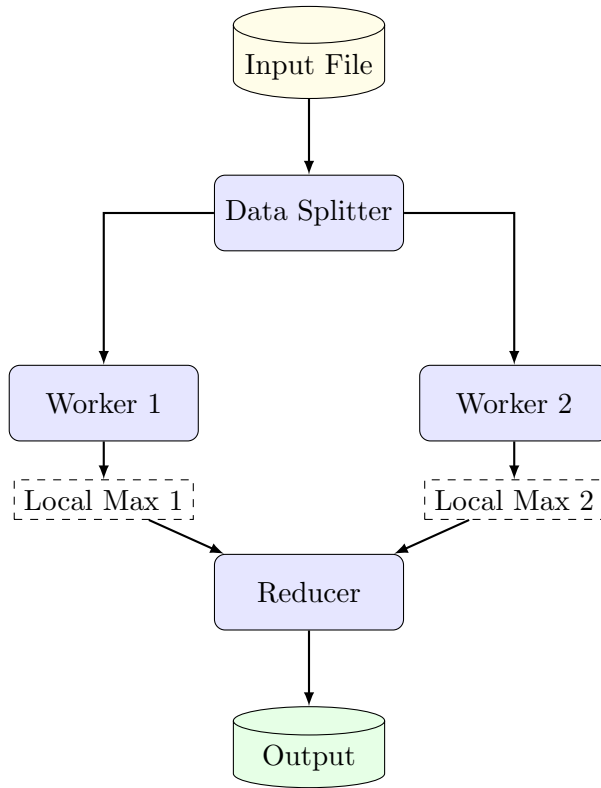


Figure 1: Top-Down Data Flow for Longest Path MapReduce

| Member   | Role      | Responsibilities                                        |
|----------|-----------|---------------------------------------------------------|
| Member 1 | Architect | Designed the process pool strategy and class structure. |
| Member 2 | Developer | Implemented the Mapper and Reducer logic.               |
| Member 3 | Analyst   | Created the architecture diagram and documentation.     |

Table 1: Group Roles and Responsibilities